

TÓPICO 02 — PRIMEIRA ETAPA

Arquitetura da Web

URI, HTTP, DNS e TLS — os pilares técnicos que sustentam a Web

O que é a arquitetura da Web?

A analogia dos Correios

Para uma carta chegar ao destino certo, existe todo um sistema por trás: um endereço padronizado, agências que recebem e despacham, veículos que transportam, e regras de como embalar e selar.

A arquitetura da Web é exatamente isso, só que para informação digital: um conjunto de tecnologias e padrões (URI, HTTP, DNS, TLS) que garante que qualquer pedido, de qualquer lugar do mundo, chegue ao servidor certo e volte com a resposta certa.

Em outras palavras

Permite comunicação entre pessoas, sistemas e serviços no mundo todo — não só entre você e um site, mas entre sistemas conversando com outros sistemas.

É baseada em protocolos de rede (regras de comunicação) e em documentos interligados por links.

É a mesma arquitetura, seja você acessando um blog simples ou um banco fazendo milhões de transações por segundo.

Objetivos

Entender os pilares da Web

URI, HTTP, DNS e TLS — as quatro tecnologias que, juntas, fazem qualquer página carregar.

Compreender o modelo cliente-servidor

Quem pede o quê, e quem responde o quê, em toda comunicação na Web.

Ver como funciona uma requisição HTTP

Do clique do usuário até o conteúdo aparecer na tela — passo a passo.

Entender por que usamos TLS (HTTPS)

O que garante que ninguém no meio do caminho consiga ler ou alterar os dados.

Diferenciar servidor Web e servidor de aplicação

Duas peças que parecem iguais, mas têm papéis bem diferentes num sistema real.

Fundamentos da Arquitetura da Web

A Web foi projetada, desde o início, para ter três qualidades:

Simple

Fácil de implementar e entender — qualquer computador pode participar da Web sem depender de tecnologia exclusiva.

Universal

Funciona em qualquer dispositivo, sistema operacional ou localização geográfica, sem exigir tradução ou adaptação especial.

Escalável

Suporta desde um blog pessoal até bilhões de acessos simultâneos, sem precisar trocar a arquitetura de base.

E tudo isso é possível graças a quatro recursos técnicos, que vamos destrinchar um a um nesta aula:

URI

HTTP

DNS

TLS

O modelo Cliente-Servidor

CLIENTE

Normalmente o navegador (mas pode ser um app de celular, ou até outro sistema).

É quem inicia a conversa: faz a requisição, esperando uma resposta.

Cada recurso pedido é identificado por um URI — o endereço único daquele recurso na Web.

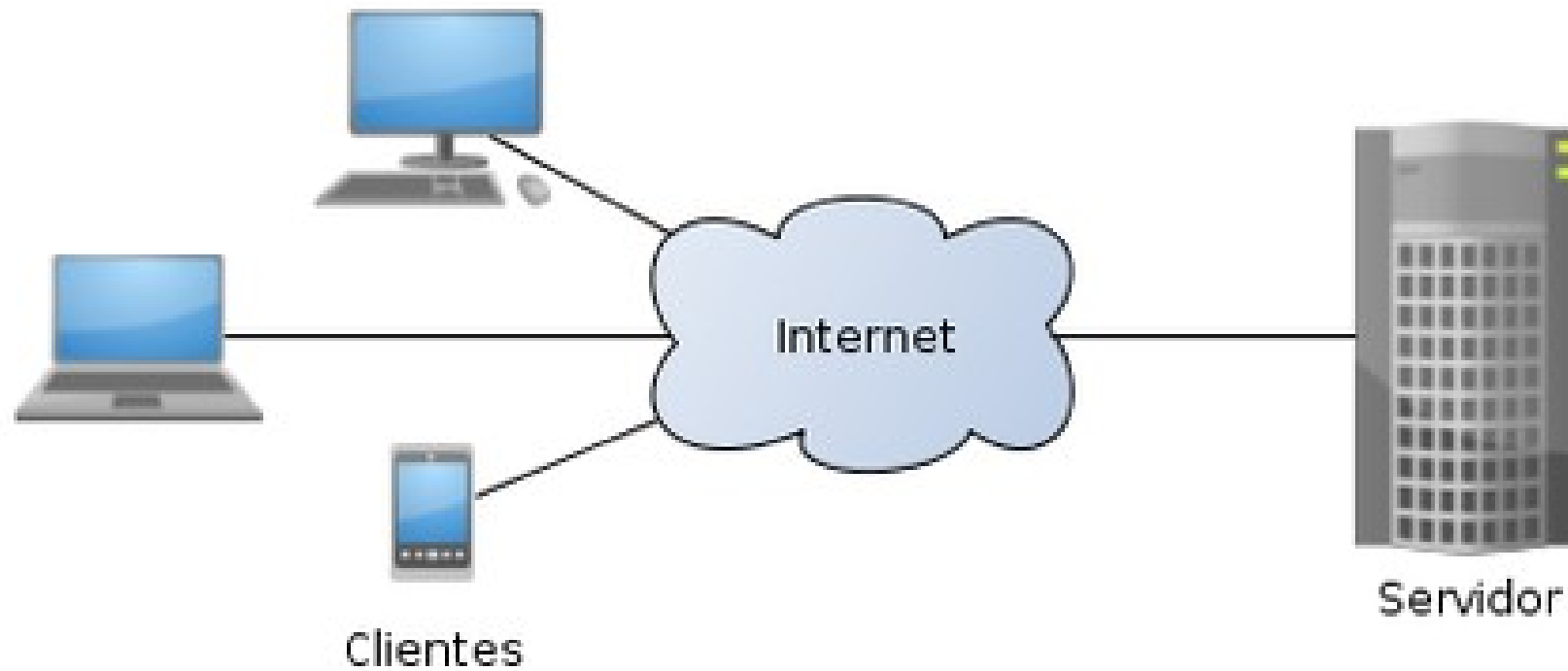
SERVIDOR

Recebe a requisição, processa (aplica regras, consulta dados) e responde.

A comunicação entre cliente e servidor acontece usando o protocolo HTTP, que por sua vez roda sobre o TCP/IP — a camada de rede que garante que os dados realmente cheguem ao destino, em ordem.

É a base de praticamente toda a arquitetura da Web.

O modelo Cliente-Servidor



DNS (Domain Name System)

A "lista telefônica" da Internet

Computadores se encontram na rede através de números — endereços IP, como 192.0.2.10. Só que números são difíceis de lembrar.

O DNS resolve esse problema: ele traduz nomes fáceis de lembrar (como exemplo.com) para o endereço IP real do servidor.

Sem o DNS, você teria que memorizar uma sequência de números para acessar cada site que usa no dia a dia.

Na prática

```
exemplo.com → 2.2.2.2
```

Toda vez que você digita um endereço no navegador, antes mesmo da requisição HTTP ser enviada, o computador consulta o DNS para descobrir qual IP corresponde àquele nome — e só então a conversa com o servidor começa.

DNS (Domain Name System)

```
daniel@pwd:~$ ping www.satc.edu.br  
PING www.satc.edu.br (177.54.50.197) 56(84) bytes of data.  
□
```


HTTP (HyperText Transfer Protocol)

HTTP é o protocolo que define exatamente como cliente e servidor conversam: o formato da mensagem de pedido e o formato da mensagem de resposta.

```
GET /index.html HTTP/1.1  
Host: exemplo.com
```

Esse é o pedido mais simples possível: "me dê o arquivo index.html, do host exemplo.com".

O que toda requisição/resposta HTTP tem

Modelo requisição → resposta: o cliente sempre inicia, o servidor sempre responde.

Métodos: GET, POST, PUT, DELETE, entre outros — cada um com uma intenção diferente.

Códigos de status: números como 200, 404 ou 500, que dizem o que aconteceu com o pedido.

Os métodos HTTP e sua intenção

GET

Buscar/ler um recurso

GET /produtos/42 → devolve os dados do produto 42, sem alterar nada.

POST

Criar um novo recurso

POST /produtos → cria um novo produto com os dados enviados no corpo da requisição.

PUT

Atualizar um recurso existente

PUT /produtos/42 → substitui os dados do produto 42 pelos novos dados enviados.

DELETE

Remover um recurso

DELETE /produtos/42 → remove o produto 42 do sistema.

Códigos de status: o que a resposta está dizendo

2xx

Sucesso

200 OK — o pedido funcionou como esperado.

3xx

Redirecionamento

301 Moved — o recurso foi movido para outro endereço.

4xx

Erro do cliente

404 Not Found — o recurso pedido não existe.

5xx

Erro do servidor

500 Internal Server Error — algo quebrou no servidor.

TLS (Transport Layer Security)

HTTP, por si só, envia tudo em texto puro — qualquer pessoa no caminho poderia ler ou alterar os dados. O TLS resolve isso, garantindo três coisas:

Confidencialidade

Os dados são criptografados — mesmo que alguém intercepte a comunicação, não consegue ler o conteúdo.

Integridade

Qualquer alteração nos dados durante o trajeto é detectada — o destinatário sabe se algo foi modificado no caminho.

Autenticidade

Confirma com quem você realmente está falando, através de um certificado digital — evitando que um site falso se passe pelo verdadeiro.

HTTPS = HTTP + TLS

$$\text{HTTP} + \text{TLS} = \text{HTTPS (https://)}$$

Handshake TLS — como a conexão segura é estabelecida, em 4 passos:

1 Cliente envia informações iniciais

Quais algoritmos de criptografia suporta, e um número aleatório.

2 Servidor responde

Com seu próprio número aleatório, seu certificado digital, e o algoritmo escolhido.

3 Cliente verifica o certificado

Confirma que o certificado é válido e realmente pertence ao domínio acessado.

4 Chave de sessão é criada

Cliente e servidor geram uma chave compartilhada e passam a criptografar toda a comunicação.

O fluxo completo de uma requisição

1

Você digita ou clica em um endereço (URI).

2

O domínio é resolvido via DNS → IP do servidor.

3

Cliente abre uma conexão TCP com o servidor (porta 80 ou 443).

4

Se for HTTPS, acontece o handshake TLS. (Conexão Segura)

5

Cliente envia a requisição HTTP.

6

Servidor processa a requisição.

7

Servidor envia a resposta HTTP, com status e conteúdo.

8

Cliente recebe a resposta e renderiza a página.

9

A conexão pode ser fechada ou mantida aberta (keep-alive).

10

Novas requisições podem seguir o mesmo fluxo.

Uma requisição e resposta HTTP, de verdade

REQUISIÇÃO

```
GET /index.html HTTP/1.1
Host: exemplo.com
```

RESPOSTA

```
HTTP/1.1 200 OK
Content-Type: text/html

<html>
  <body><h1>Olá, Mundo!</h1></body>
</html>
```

Lendo a resposta, parte por parte

200 OK

O status: a requisição foi bem-sucedida.

Content-Type: text/html

Avisa ao navegador que o conteúdo a seguir é HTML, para que ele saiba como interpretar.

O corpo (<html>...</html>)

É o conteúdo em si — o HTML que o navegador vai transformar na página visível.

Conexões persistentes (keep-alive)

Abrir uma nova conexão TCP para cada requisição tem um custo (tempo e processamento). O HTTP/1.1 resolve isso permitindo reaproveitar a mesma conexão para várias requisições. **Abrir uma nova conexão TCP para cada requisição é como ligar, falar uma frase, desligar e ligar novamente para a próxima frase. Cada ligação leva um tempo para ser estabelecida. Reaproveitar a conexão é como manter a ligação aberta e continuar conversando. Você não precisa fazer uma nova chamada a cada frase.**

```
GET /index.html HTTP/1.1
Host: exemplo.com
Connection: keep-alive
```

Menos latência e Uso de CPU

Sem precisar refazer o processo de abrir conexão (e handshake TLS) a cada requisição, usando menos CPU do servidor, com menos utilização de rede e carregando informações de sistemas mais rápido.

Por exemplo, no Node.js:

```
</> JavaScript
const https = require('https');

const agent = new https.Agent({
  keepAlive: true
});
```

Servidor decide o tempo

O servidor pode manter a conexão aberta por um tempo configurável, aguardando novas requisições do mesmo cliente.

```
</> Nginx

keepalive_timeout 65;
keepalive_requests 1000;
```


Vendo o HTTP funcionar no seu computador, agora

Opção 1 — Terminal (curl)

```
curl -v https://www.satc.edu.br
```

O parâmetro -v ("verbose") mostra, na tela, todo o processo: a resolução DNS, o handshake TLS, a requisição enviada e a resposta completa recebida — exatamente os passos que vimos nesta aula, acontecendo de verdade.

Opção 2 — DevTools do navegador

1. Abra qualquer site e pressione F12 (ou clique com o botão direito → Inspecionar).
2. Clique na aba "Network" (Rede) e recarregue a página.
3. Clique em qualquer requisição listada: você verá o método, os headers, o status code e o corpo da resposta.

A infraestrutura por trás de um site grande

Um blog pessoal pode rodar em um único servidor. Mas um sistema com milhões de usuários precisa de uma infraestrutura

Data centers

Prédios especializados, com múltiplos servidores, energia redundante e conexões de altíssima velocidade.

Load balancer (balanceador de carga)

Distribui as requisições que chegam entre vários servidores, para nenhum deles ficar sobrecarregado.

Cache / CDN

Guarda cópias de conteúdo mais próximas do usuário final, reduzindo o tempo de resposta.

Microserviços e cluster de banco de dados

Em vez de um sistema único e gigante, várias partes menores e independentes, cada uma com sua responsabilidade, e o banco de dados replicado em vários servidores.

Servidor Web x Servidor de Aplicação

Servidor Web

Software que processa requisições HTTP e entrega:

- Arquivos estáticos (HTML, CSS, JS, imagens)
- Respostas geradas por aplicações

Componentes típicos: processamento de requisições, gerenciamento de recursos, segurança (autenticação/autorização) e logs.

Exemplos: Apache, Nginx, IIS.

Servidor de Aplicação

Ambiente para executar aplicações web dinâmicas, focado na lógica de negócio.

Faz a ponte entre: requisições HTTP ↔ código da aplicação
↔ banco de dados e outros serviços.

É aqui que vive o código que você vai escrever nesta disciplina — as regras de negócio da sua API.

Exemplos de uso: APIs REST, aplicações corporativas, servidores de jogos.

Frameworks: onde tudo isso se torna prático

★ Java / Spring Boot

É o que usaremos nesta disciplina. O Spring Boot já traz um servidor de aplicação embutido (o Tomcat), então quando você "roda" sua API Java, na prática já existe um servidor HTTP funcionando por trás, esperando requisições — tudo que vimos nesta aula, funcionando dentro do seu próprio código.

Python / Django, Flask

Executados em servidores WSGI/ASGI, como Werkzeug, Gunicorn ou Uvicorn.

Node.js / Express.js

O próprio Node.js atua diretamente como servidor de aplicação, sem precisar de uma peça separada.

Roadmap de habilidades para 2026

1

Lógica de programação

A base de tudo: entender como decompor um problema em passos que o computador executa.

2

Uma linguagem backend (Java)

A linguagem que usaremos na disciplina — muito usada no mercado corporativo brasileiro.

3

Banco de dados

Saber guardar, consultar e organizar informações de forma eficiente e segura.

4

APIs e arquitetura REST

O padrão mais usado hoje para sistemas conversarem entre si.

5

Versionamento com Git

Controlar e colaborar em código sem perder histórico ou sobrescrever o trabalho de outros.

6

Nuvem (Cloud) e infraestrutura

Onde e como os sistemas realmente rodam no mercado hoje — Google Cloud, AWS, Azure.

O papel do desenvolvedor Backend hoje

Constrói e mantém APIs

As "portas de entrada" que permitem que apps e sites conversem com os sistemas da empresa.

Modela e cuida dos dados

Decide como as informações são organizadas e guardadas nos bancos de dados, garantindo consistência.

Garante segurança e performance

Protege dados sensíveis e garante que o sistema responda rápido mesmo com muitos usuários simultâneos.

Integra sistemas diferentes

Conecta o sistema da empresa a serviços externos: pagamento, envio de e-mail, IA, logística.

ATAQUES CONHECIDOS

1 Caso 1 – XZ Utils (2024)

O atacante passou cerca de 2 anos contribuindo legitimamente com o projeto open source (Linux), corrigindo bugs e conquistando a confiança da comunidade até se tornar mantenedor.

Com esse nível de acesso, inseriu uma backdoor extremamente sofisticada em uma nova versão da biblioteca, que poderia comprometer o serviço de acesso remoto (SSH) em sistemas Linux.

O ataque só foi descoberto porque o engenheiro de software Andres Freund (Microsoft) percebeu um aumento incomum no consumo de CPU e uma pequena lentidão durante autenticações SSH. Ao investigar esse comportamento inesperado, encontrou a backdoor antes que ela fosse amplamente distribuída

2

2 Caso 2 – event-stream (2018)

O atacante identificou uma biblioteca open source popular, utilizada por milhares de aplicações, e se aproximou do mantenedor oferecendo ajuda no projeto. Após conquistar a confiança do responsável, recebeu permissão para publicar novas versões do pacote.

Em uma atualização, inseriu uma dependência maliciosa que continha código capaz de comprometer aplicações específicas relacionadas ao mercado de criptomoedas. O código malicioso ficou escondido em uma dependência secundária e só era executado em determinadas condições, dificultando sua identificação. O problema foi descoberto quando pesquisadores analisaram o comportamento da nova versão e perceberam alterações suspeitas no pacote.

2

ATAQUES CONHECIDOS – DICA BACKEND

A segurança não depende apenas do seu código, mas também da confiança depositada em cada componente que você utiliza.

A informação que você expõe pode ser o mapa que o atacante precisa para encontrar o caminho até sua aplicação.

REFERENCIAS

BERNERS-LEE, T.; FISCHETTI, M. Weaving the Web: The Original Design and Ultimate Destiny of the World Wide Web by Its Inventor. San Francisco: Harper, 2000.

BUSH, V. As We May Think. The Atlantic Monthly, v. 176, n. 1, p. 101–108, 1945.

Tim O'Reilly. What Is Web 2.0: Design Patterns and Business Models for the Next Generation of Software. O'Reilly Media, 2005.

World Wide Web Consortium. World Wide Web Consortium (W3C). Disponível em: <https://www.w3.org/>. Acesso em: 3 ago. 2026.

Mozilla Foundation. MDN Web Docs. Disponível em: <https://developer.mozilla.org/>. Acesso em: 3 ago. 2026.

RFC 9110 – HTTP Semantics. Internet Engineering Task Force (IETF), 2022.

RFC 9112 – HTTP/1.1. Internet Engineering Task Force (IETF), 2022.

RFC 9113 – HTTP/2. Internet Engineering Task Force (IETF), 2022.

RFC 9114 – HTTP/3. Internet Engineering Task Force (IETF), 2022.

Backend Roadmap. Developer Roadmaps. Disponível em: <https://roadmap.sh/backend>. Acesso em: 3 ago. 2026.

RESUMO DA AULA

O que levamos de hoje

- URI identifica o recurso; DNS traduz o nome em IP; HTTP é a língua da conversa; TLS garante segurança.
- O fluxo completo de uma requisição envolve DNS → TCP → (TLS) → requisição HTTP → resposta HTTP.
- Métodos HTTP (GET/POST/PUT/DELETE) expressam a intenção; status codes (2xx-5xx) dizem o que aconteceu.
- Servidor Web entrega conteúdo; Servidor de Aplicação roda a lógica de negócio — é onde seu código vai viver.

PRÓXIMA AULA — AMBIENTE DE DESENVOLVIMENTO

Vamos instalar e configurar as ferramentas que usaremos durante toda a disciplina: JDK, IDE e Git.